



Abschlussprüfung Frühjahr 2026

Fachinformatiker für Anwendungsentwicklung
Dokumentation zur betrieblichen Projektarbeit

Festival Catering-App

**Implementierung einer App für Festival-Verpflegung mit
Vorbestellung & dynamischer Abholung**

Abgabetermin: Saarbrücken, den t.b.d.

Prüfungsbewerber:

Hendrik Wolf, Leon Horch, Said Dahdouh
Am Mügelsberg 1
66111 Saarbrücken



Ausbildungsbetrieb:

TGBBZ1
Am Mügelsberg 1
66111 Saarbrücken

Inhaltsverzeichnis

Abbildungsverzeichnis	III
Tabellenverzeichnis	IV
Abkürzungsverzeichnis	V
1 Einleitung	1
1.1 Projektbeschreibung	1
1.2 Projektziel	1
1.3 Projektumfeld	2
1.4 Projektbegründung	2
2 Projektplanung	2
2.1 Projektphasen	2
2.2 Ressourcenplanung	3
2.3 Entwicklungsprozess	3
3 Analysephase	4
3.1 Ist-Analyse	4
3.2 Soll-Konzept	4
3.2.1 Benutzersicht	5
3.2.2 Standsicht	5
3.2.3 Bestands- und Bestellverwaltung	5
3.2.4 Sonderfälle	5
3.2.5 Zielsetzung des Soll-Konzepts	5
3.3 Wirtschaftlichkeitsanalyse	6
3.3.1 „Make or Buy“-Entscheidung	6
3.3.2 Projektkosten	6
3.3.3 Nutzen / Einsparungen	6
3.3.4 Amortisationsdauer	7
3.4 Anwendungsfälle	7
3.5 Qualitätsanforderungen	7
3.5.1 Funktionale Qualität	8
3.5.2 Benutzerfreundlichkeit	8
3.5.3 Performance	8
3.5.4 Zuverlässigkeit und Stabilität	8
3.5.5 Nachvollziehbarkeit	8
3.5.6 Erweiterbarkeit	8
3.6 Lastenheft/Fachkonzept	8
4 Entwurfsphase	9



4.1	Zielformat	9
4.2	Architekturdesign	9
4.3	Entwurf der Benutzeroberfläche	10
4.4	Datenmodell	11
4.5	Geschäftslogik	11
4.6	Maßnahmen zur Qualitätssicherung	12
4.7	Pflichtenheft	12
5	Implementierungsphase	12
5.1	Implementierung der Datenstrukturen	12
5.2	Implementierung der Geschäftslogik	13
5.3	Implementierung der Benutzeroberfläche	13
6	Abnahmephase	13
7	Dokumentation	14
8	Fazit	14
8.1	Soll-/Ist-Vergleich	14
8.2	Lessons Learned	14
8.3	Ausblick	15
A	Anhang	i
A.1	Verwendete Ressourcen	i
A.2	Lastenheft (Auszug)	ii
A.2.1	Funktionale Anforderungen	ii
A.2.2	Nichtfunktionale Anforderungen	iii
A.3	Use-Case-Diagramm	v
A.4	Mockups	vi
A.5	ER-Diagramm	xii
A.6	Relationales Datenbankmodell	xiii
A.7	Klassendiagramm	xiv
A.8	Pflichtenheft (Auszug)	xv
A.8.1	Umsetzung Funktionale Anforderungen	xv
A.8.2	Umsetzung Nichtfunktionale Anforderungen	xvii
A.9	Benutzerhandbuch	xix



Abbildungsverzeichnis

1	Use-Case-Diagramm	v
2	Login-Seite	vi
3	Besucher-Seite	vii
4	Bestellungs-Seite	viii
5	Einzelanzeige Bestellung	ix
6	Guthaben Popup	x
7	Verkäufer-Seite	xi
8	ER-Diagramm	xii
9	Relationales Datenbankmodell	xiii
10	Klassendiagramm	xiv



Tabellenverzeichnis

1	Zeitplanung	3
2	Kostenaufstellung	6



Abkürzungsverzeichnis

ODBC	Open Database Connectivity
MVC	Model-View-Controller
GUI	Graphical User Interface
SQL	Structured Query Language
PIL	Python Imaging Library
QR	Quick-Response (Code)
VIP	Very Important Person
KI	Künstliche Intelligenz



1 Einleitung

Die folgende Projektdokumentation schildert den Ablauf des Demo-Abschlussprojektes, welches die Autoren im Rahmen ihrer Ausbildung zum Fachinformatiker Fachrichtung Anwendungsentwicklung durchgeführt haben. Auftraggeber ist das TGBBZ1, eine Berufsschule in Saarbrücken. Das Projekt dient zur Vorbereitung auf die IHK-Abschlussprüfung.

1.1 Projektbeschreibung

Im Rahmen des Projekts wird ein digitales Bestell- und Abholsystem für ein Musikfestival konzipiert und umgesetzt. Ziel ist es, den klassischen Verkaufsprozess an Essens- und Getränkeständen durch eine softwaregestützte Lösung zu optimieren. Besucher*innen des Festivals erhalten die Möglichkeit, über eine mobile Anwendung Bestellungen aufzugeben. Nach der Anmeldung mittels Ticketnummer kann ein Guthaben verwaltet werden, welches für Bestellungen genutzt wird. Die Bestellung erfolgt standgebunden und berücksichtigt aktuelle Verfügbarkeiten sowie geschätzte Wartezeiten. Pro Ticket wird ein digitaler Abholcode (QR-Code) generiert, um beim Abholen die Übergabe zu verifizieren. Der Status der Bestellung wird in Echtzeit aktualisiert (z.B. „In Bearbeitung, „Bereit zur Abholung“). Das Standpersonal kann Bestellungen verwalten, deren Status anpassen sowie Produkt-Bestände nachfüllen. Die Ausgabe erfolgt durch das Scannen des QR-Codes. Zusätzlich beinhaltet das System Funktionen zur Rückerstattung bei stornierten Bestellungen und stellt eine priorisierte Behandlung von VIP-Besucher*innen bereit.

1.2 Projektziel

Ziel des Projekts ist die Entwicklung einer Anwendung zur Digitalisierung und Optimierung der Bestell- und Ausgabeprozesse auf einem Musikfestival.

Dabei sollen insbesondere folgende Ziele erreicht werden:

- Reduzierung von Warteschlangen an Verkaufsständen
- Verbesserung der Benutzerfreundlichkeit für Besucher*innen
- Effizientere Organisation der Bestellabläufe für Standpersonal
- Transparente Darstellung von Bestellstatus und Wartezeiten
- Echtzeit-Verwaltung von Beständen und Bestellungen

Das System soll mehrere Benutzerrollen (Besucher, VIP-Besucher, Standpersonal) unterstützen und deren spezifische Anforderungen abbilden.

1.3 Projektumfeld

Das Projekt wird im Rahmen eines Abschlussprojekts zum Fachinformatiker für Anwendungsentwicklung durchgeführt. Auftraggeber ist das TGBBZ1, das als fiktiver Kunde agiert.

Die Entwicklung erfolgt in einer simulierten, aber realitätsnahen Projektumgebung, welche reale Bedingungen eines Musikfestivals nachbildet. Dazu zählen:

- Mehrere Verkaufsstände mit unterschiedlichen Angeboten
- Zugriff durch Personen mit unterschiedlichen Benutzerrollen
- Echtzeitkommunikation zwischen Frontend und Backend

Die Umsetzung erfolgt als Gruppenprojekt unter Verwendung moderner Entwicklungswerkzeuge und -methoden. Die Anwendung besteht aus mehreren Komponenten, wie z.B. der Frontend-App (Python, Tkinter) und einer Datenbank, umgesetzt über Microsoft SQL Server.

1.4 Projektbegründung

Auf Musikfestivals entstehen häufig lange Warteschlangen an Essens- und Getränkeständen, was zu Unzufriedenheit bei Besucher*innen und chaotischen Abläufen führt. Durch die Digitalisierung des Bestellprozesses können Warteschlangen durch zeitversetzte Abholung vermieden, Stoßzeiten durch bessere Planbarkeit entzerrt und Kommunikationsfehler bei Bestellungen reduziert werden. Die Verwaltung von Echtzeitdaten hilft bei der Optimierung des Lager- und Bestandnachfüllprozesses. Das Projekt bietet zudem einen hohen Praxisbezug, da vergleichbare Systeme bereits in der Gastronomie und auf Veranstaltungen eingesetzt werden.

2 Projektplanung

2.1 Projektphasen

Für die Umsetzung des Projektes standen den Autoren 70 Schulstunden zur Verfügung. Diese wurden vor Projektbeginn auf verschiedene Phasen verteilt, die während des Entwicklungsprozesses durchlaufen werden. Da das Projektthema aus Lernfeld 10 (Blockwoche 1) fortgeführt wird, befindet sich das Projekt zu diesem Zeitpunkt bereits im Entwicklungsprozess. Aus der [Tabelle 1: Zeitplanung](#) kann deshalb eine Zeitplanung für die noch zu erledigenden Aufgaben ab Beginn dieses Projektes entnommen werden.

Blockwoche	Leon	Said	Hendrik
10 / 4 (6h)	Datenbank aufsetzen (2h) Datenbank befüllen (2h)		Zeitplan (1h) Datenbank befüllen (2h)
10 / 5 (8h)	Datenbeschaffung umstellen (2h) Diagramm erstellen (4h)	Use-Cases (2h) Diagramm erstellen (4h)	Datenbeschaffung umstellen (2h) Diagramm erstellen (4h)
10 / 6 (12h)	Update der Bestellungszeit (6h) Bestellung durch Mitarbeiter (4h)	Guthaben-aufladen Popup (4h) Fehlerbehandlung wenn zu wenig Guthaben (4h)	Priorisieren für VIPs implementieren (4h) Nachrichtenklasse + Aktualisierung (6h)
12 / 1 (16h)	Dokumentation unterstützen (4h) Skalierung optional (8h)	Dokumentation aufsetzen (12h)	Dokumentation unterstützen (4h) Freigabe an Freunde erweitern (2h) Skalierung optional (8h)
12 / 2 (14h)	Peer-Review Testgruppe (6h) Präsentation vorbereiten (8h)	Peer-Review Testgruppe (6h) Präsentation vorbereiten (8h)	Peer-Review Testgruppe (6h) Präsentation vorbereiten (8h)
12 / 3 (14h)	Pufferzeit	Pufferzeit	Pufferzeit
Gesamt 70h			

Tabelle 1: Zeitplanung

2.2 Ressourcenplanung

Eine detaillierte Übersicht der verwendeten Ressourcen lässt sich im Anhang [A.1: Verwendete Ressourcen](#) nachlesen. Dort sind alle Ressourcen aufgelistet, welche für das Projekt eingesetzt wurden. Das beinhaltet Hard- und Softwareressourcen sowie Personal. Bei der Auswahl der verwendeten Software wurde darauf geachtet, dass diese kostenfrei zur Verfügung steht oder uns bereits Lizenzen bereit gestellt waren.

2.3 Entwicklungsprozess

Für die Umsetzung des Projekts wurde ein an das Wasserfallmodell angelehnter Entwicklungsprozess gewählt. Dieses Vorgehensmodell zeichnet sich durch eine klare, sequentielle Struktur der einzelnen Projektphasen aus und eignet sich insbesondere für Projekte mit weitgehend stabilen Anforderungen.

Zu Beginn des Projekts erfolgte die Analyse der Ausgangssituation sowie die Definition der Anforderungen. Auf dieser Grundlage wurde ein Soll-Konzept erarbeitet, das als Basis für die anschließende Umsetzung diente.

Im weiteren Verlauf wurde die Implementierung der Anwendung durchgeführt. Dabei wurden die zuvor definierten Anforderungen schrittweise umgesetzt. Aufgrund des schulischen Projekts

tumfelds erfolgte die Entwicklung überwiegend ohne formale Iterationszyklen, wobei kleinere Anpassungen und Korrekturen während der Implementierungsphase berücksichtigt wurden.

Nach der Implementierung wurden grundlegende Funktionstests durchgeführt, um die korrekte Funktionsweise der Anwendung sicherzustellen. Hierbei lag der Fokus insbesondere auf der Überprüfung zentraler Prozesse wie der Bestellung, der Statusverwaltung und der Bestandsaktualisierung.

Das gewählte Vorgehensmodell ermöglichte eine strukturierte Bearbeitung des Projekts und eine klare Trennung der einzelnen Projektphasen. Gleichzeitig zeigte sich, dass bei zukünftigen Projekten mit dynamischeren Anforderungen auch iterative Vorgehensmodelle, wie beispielsweise agile Methoden, in Betracht gezogen werden könnten.

3 Analysephase

3.1 Ist-Analyse

Auf Musikfestivals erfolgt der Verkauf von Speisen und Getränken in der Regel ausschließlich vor Ort an einzelnen Verkaufsständen. Der typische Ablauf gestaltet sich wie folgt:

- Besucher*innen stellen sich vor Ort an einem Stand an
- Bestellung wird direkt beim Personal aufgegeben
- Bezahlung erfolgt unmittelbar (bar oder per Karte)
- Bestellung wird anschließend zubereitet
- Besucher*innen warten an der Ausgabe
- Übergabe erfolgt direkt am Stand

Dieser Prozess findet sequenziell statt, wodurch sich insbesondere zu Stoßzeiten lange Warteschlangen bilden. Das Standpersonal muss gleichzeitig Bestellungen entgegennehmen, kassieren, zubereiten und ausgeben, was zu Stresssituationen und erhöhter Fehleranfälligkeit führt. Besucher*innen haben momentan keine direkte Information über die aktuellen Wartezeiten, die Verfügbarkeit von Produkten oder den Status ihrer Bestellung.

3.2 Soll-Konzept

Zur Behebung der in der Ist-Analyse beschriebenen Schwächen soll ein digitales Bestell- und Abholsystem entwickelt werden, das die bisherigen manuellen Abläufe auf dem Festival teilweise ersetzt und organisatorisch verbessert.



3.2.1 Benutzersicht

Besucher*innen sollen sich über ihre Ticketnummer am System anmelden können. Nach erfolgreicher Anmeldung soll ein Ticket-gebundenes Guthaben zur Verfügung stehen, das für Bestellungen genutzt werden kann. Außerdem soll dem Benutzer bzw. der Benutzerin ein QR-Code zur Verfügung gestellt werden, der zur Abholung der Bestellung verwendet wird. Innerhalb der Anwendung sollen Produkte standbezogen ausgewählt und bestellt werden können. Zusätzlich sollen Informationen über Wartezeiten und Produktverfügbarkeiten angezeigt werden. Ebenso soll der aktuelle Bearbeitungsstatus der Bestellung jederzeit einsehbar sein.

3.2.2 Standsicht

Für das Standpersonal soll eine Verwaltungsoberfläche bereitgestellt werden, in der eingehende Bestellungen übersichtlich dargestellt werden. Die Bestellungen sollen dort nach Bearbeitungsstatus und gegebenenfalls Priorität verwaltet werden können. Darüber hinaus soll das Personal den Status einzelner Bestellungen ändern sowie Produktbestände einsehen und bei Bedarf aktualisieren können.

3.2.3 Bestands- und Bestellverwaltung

Das System soll Produktbestände zentral verwalten. Bestellungen dürfen nur dann abgeschlossen werden, wenn die benötigten Produkte in ausreichender Menge verfügbar sind. Nach Abschluss einer Bestellung oder Ausgabe soll der Bestand automatisch angepasst werden.

3.2.4 Sonderfälle

Es soll eine Möglichkeit zum stornieren von Bestellungen bereit gestellt werden, darauf folgend muss das zuvor belastete Guthaben automatisch zurückgebucht werden. Zusätzlich soll das System eine bevorzugte Behandlung von VIP-Besucher*innen ermöglichen.

3.2.5 Zielsetzung des Soll-Konzepts

Durch die Umsetzung des Soll-Konzepts sollen Wartezeiten reduziert, Abläufe an den Verkaufsständen strukturiert und die Transparenz für alle beteiligten Benutzergruppen erhöht werden. Gleichzeitig soll die digitale Verarbeitung der Bestellungen eine bessere Nachvollziehbarkeit und effizientere Bestandsverwaltung ermöglichen.

3.3 Wirtschaftlichkeitsanalyse

Ziel der Wirtschaftlichkeitsanalyse ist es, die finanzielle Vorteilhaftigkeit der entwickelten Anwendung zu bewerten. Da es sich um ein Schulprojekt handelt, basieren die folgenden Berechnungen auf realistischen Annahmen.

3.3.1 „Make or Buy“-Entscheidung

Im Vorfeld der Entwicklung wurde geprüft, ob eine bestehende Softwarelösung eingesetzt werden kann (Buy) oder eine Eigenentwicklung (Make) sinnvoll ist. Im Rahmen dieses Projekts wurde die Eigenentwicklung gewählt, da die Anforderungen sehr spezifisch sind und eine flexible Erweiterung des Systems ermöglicht werden soll.

3.3.2 Projektkosten

Die Projektkosten, die während der Entwicklung des Projektes anfallen, sollen im Folgenden kalkuliert werden. Dafür müssen neben den Personalkosten, die durch die Realisierung des Projektes verursacht werden, auch noch die groben Aufwendungen für die Ressourcen berücksichtigt werden. Im Sinne dieses Projektes haben wir nun mit einem Stundensatz von 8€ für einen Schüler und einem Stundensatz von 30€ für einen Lehrer gerechnet.

Die Kosten, die wir für die jeweiligen Vorgänge des Projektes berechnet haben, sowie die gesamten Projektkosten lassen sich der Tabelle 2: [Kostenaufstellung](#) entnehmen.

Vorgang	Mitarbeiter	Zeit	Personal	Ressourcen	Gesamt
Entwicklungskosten	3 x Schüler	64h	1536,00 €	500,00 €	2036,00 €
Peer-Review	3 x Schüler	6h	144,00 €	30,00 €	174,00 €
Abnahme / Fachgespräch	3 x Lehrer	2h	180,00 €	20,00 €	200,00 €
Projektkosten gesamt					2.410,00 €

Tabelle 2: Kostenaufstellung

3.3.3 Nutzen / Einsparungen

Annahmen:

- 15.000 Besucher pro Tag
- 60 % nutzen das System → 9.000 Bestellungen
- 2 € Mehrumsatz pro Bestellung

Berechnung:

$$9.000 \times 2 \text{ €} = 18.000 \text{ € Mehrumsatz pro Tag}$$

Zusätzliche wirtschaftliche Vorteile:

- Reduzierter Personalbedarf → Einsparung von Personalkosten
- Bessere Planbarkeit → weniger Lebensmittelverschwendung
- Schnellere Abwicklung → mehr Verkäufe möglich
- Höhere Kundenzufriedenheit → langfristige Umsatzsteigerung

3.3.4 Amortisationsdauer

Die Amortisationsdauer beschreibt den Zeitraum, nach dem sich die Investitionskosten durch den generierten Mehrwert ausgeglichen haben.

Bei Gesamtprojektkosten in Höhe von 2.410 € und einem geschätzten täglichen Mehrumsatz von 18.000 € ergibt sich folgende Berechnung:

$$2.410 \text{ €} / 18.000 \text{ €} = 0,134 \text{ Tage}$$

Dies entspricht einer Amortisationsdauer von ca. 3,2 Stunden.

Somit amortisiert sich das System bereits innerhalb weniger Stunden am ersten Veranstaltungstag. Selbst bei deutlich geringeren Nutzerzahlen oder Umsätzen ist von einer sehr schnellen Refinanzierung auszugehen.

3.4 Anwendungsfälle

Im Laufe der Analysephase wurde ein Use-Case-Diagramm erstellt, um eine ungefähre Übersicht über die Anwendungsfälle zu erhalten. Das Diagramm ist im Anhang unter [A.3: Use-Case-Diagramm](#) zu finden und bildet alle Funktionen ab, die aus Endanwendersicht benötigt werden.

3.5 Qualitätsanforderungen

Die Qualitätsanforderungen bestimmen die Erwartungen an die Qualität des Systems und dienen als Maßstab für die Bewertung der Umsetzung.

3.5.1 Funktionale Qualität

Das System muss alle definierten Funktionen vollständig und korrekt abbilden. Bestellungen dürfen nur bei ausreichendem Guthaben und verfügbaren Produkten durchgeführt werden. Statusänderungen müssen nachvollziehbar und konsistent erfolgen.

3.5.2 Benutzerfreundlichkeit

Die Anwendung soll eine einfache und verständliche Bedienung ermöglichen. Alle relevanten Informationen, wie Bestellstatus oder Wartezeiten, sollen klar und übersichtlich dargestellt werden.

3.5.3 Performance

Die Reaktionszeiten des Systems sollen so gering wie möglich gehalten werden, um eine flüssige Nutzung zu gewährleisten. Insbesondere das Aufgeben von Bestellungen und das Aktualisieren von Statusinformationen sollen ohne spürbare Verzögerung erfolgen.

3.5.4 Zuverlässigkeit und Stabilität

Das System soll auch unter hoher Last stabil funktionieren. Fehlerhafte Zustände, wie doppelte Bestellungen oder falsche Bestände, sollen vermieden werden.

3.5.5 Nachvollziehbarkeit

Alle relevanten Aktionen, insbesondere Bestellungen, Stornierungen und Statusänderungen, sollen im System nachvollziehbar gespeichert werden.

3.5.6 Erweiterbarkeit

Das System soll so konzipiert sein, dass zukünftige Erweiterungen, wie zusätzliche Benutzerrollen oder neue Funktionen, ohne grundlegende Änderungen möglich sind.

3.6 Lastenheft/Fachkonzept

Als Ergebnis der Analysephase wurde ein Lastenheft erstellt. Dieses umfasst alle Anforderungen der Auftraggeber an die zu erstellende Anwendung. Ein Auszug aus dem Lastenheft mit Fokus auf die konkreten Anforderungen befindet sich im Anhang [A.2: Lastenheft \(Auszug\)](#).

4 Entwurfsphase

4.1 Zielplattform

Im Rahmen der Entwurfsphase wurde die Zielplattform für die Umsetzung des Systems festgelegt. Dabei wurden sowohl die Anforderungen des Projekts als auch die im schulischen Umfeld verfügbaren Technologien berücksichtigt. Die Anwendung wird als Desktop-Anwendung entwickelt und basiert auf der Programmiersprache Python. Python wurde gewählt, da im schulischen Unterricht größtenteils diese Sprache verwendet wurde und diese somit eine sichere und effiziente Umsetzung ermöglicht. Zudem bietet Python eine große Anzahl an Bibliotheken, die die Entwicklung unterstützen.

Für die grafische Benutzeroberfläche wird die Bibliothek Tkinter eingesetzt. Tkinter ist Bestandteil der Standardbibliothek von Python und ermöglicht die Erstellung einfacher, plattformunabhängiger Benutzeroberflächen. Aufgrund der guten Integration in Python und der geringen Einstiegshürde eignet sich Tkinter insbesondere für den Einsatz im schulischen Kontext. Zur Datenhaltung wird ein Microsoft SQL Server verwendet. Dieser stellt ein relationales Datenbankmanagementsystem dar und ermöglicht eine strukturierte und konsistente Speicherung der benötigten Daten, wie beispielsweise Benutzerinformationen, Bestellungen und Produktbestände. Die Verwaltung der Datenbank erfolgt über das Tool SQL Server Management Studio 22. Die Kommunikation zwischen Anwendung und Datenbank wird über die Python-Bibliothek *pyodbc* realisiert. Diese ermöglicht den Zugriff auf relationale Datenbanken über eine standardisierte Schnittstelle (ODBC) und unterstützt die Ausführung von SQL-Abfragen direkt aus der Anwendung heraus.

Durch die gewählte Kombination aus Python, Tkinter und Microsoft SQL Server wird eine klare Trennung zwischen Benutzeroberfläche, Anwendungslogik und Datenhaltung erreicht. Gleichzeitig ermöglicht die Zielplattform eine stabile und nachvollziehbare Umsetzung der Anforderungen im Rahmen des Projekts.

4.2 Architekturdesign

Für den Aufbau der Anwendung wurde eine an das Model-View-Controller-Muster (MVC) angelehnte Softwarearchitektur gewählt. Ziel dieser Architektur ist die Trennung von Benutzeroberfläche, Anwendungslogik und Datenzugriff, um die Anwendung übersichtlicher, wartbarer und besser erweiterbar zu gestalten.

Das **Model** umfasst in diesem Projekt insbesondere den Zugriff auf die Datenbank sowie die Verarbeitung und Bereitstellung der benötigten Daten. Hierzu zählen unter anderem Funktionen zum Abrufen von Bestellungen, Guthaben, Benachrichtigungen und Produktbeständen sowie zum Speichern von Änderungen, beispielsweise bei Stornierungen oder Guthabenbuchungen.



Die **View** bildet die grafische Benutzeroberfläche ab. Diese wurde mit der Python-Bibliothek Tkinter umgesetzt und stellt die verschiedenen Benutzerseiten, Dialoge und Steuerelemente bereit. Dazu gehören unter anderem Tabellen zur Anzeige offener Bestellungen, Eingabefelder für Ticketnummern, Schaltflächen zur Navigation sowie die Darstellung des QR-Codes für die Abholung.

Der **Controller** übernimmt die Steuerung der Benutzerinteraktionen und verbindet die Benutzeroberfläche mit der Datenlogik. Er verarbeitet Eingaben, ruft Datenbankfunktionen auf und aktualisiert anschließend die dargestellten Inhalte in der Oberfläche. Dadurch wird sichergestellt, dass Änderungen im System, wie zum Beispiel neue Bestellungen oder Statusänderungen, direkt in der Anwendung sichtbar werden.

Die Umsetzung orientiert sich an diesem Muster, ohne es vollständig in strikt getrennte Klassenstrukturen zu überführen. Insbesondere innerhalb einzelner GUI-Klassen werden sowohl Darstellungslogik als auch steuernde Funktionen zusammengefasst. Dennoch ist eine fachliche Trennung der Verantwortlichkeiten erkennbar: Die Oberfläche dient primär der Darstellung, während Datenbankoperationen über separate Methoden und Objekte ausgeführt werden. Ein Beispiel hierfür ist die Besucheransicht. Diese stellt dem Benutzer Informationen wie Ticketnummer, Guthaben, offene Bestellungen, Benachrichtigungen und den QR-Code grafisch zur Verfügung. Gleichzeitig werden Benutzeraktionen, wie das Stornieren einer Bestellung, das Freischalten einer Bestellung für Freund*innen oder das Aufladen von Guthaben, über entsprechende Steuerungsfunktionen verarbeitet. Die dafür notwendigen Daten werden über die Datenzugriffsschicht aus der Datenbank geladen und nach Änderungen erneut in der Oberfläche dargestellt.

Durch die Wahl einer an MVC angelehnten Architektur wird die Anwendung logisch strukturiert. Dies erleichtert die Entwicklung im Projektverlauf sowie die spätere Wartung und Erweiterung der Software. Insbesondere die Trennung von Oberflächenkomponenten und Datenzugriff trägt dazu bei, Änderungen gezielter umsetzen zu können.

4.3 Entwurf der Benutzeroberfläche

Im Rahmen der Entwurfsphase wurde die Benutzeroberfläche der Anwendung zunächst in Form von Mockups geplant. Ziel dieser Vorab-Visualisierung war es, die Struktur, Navigation und grundlegende Bedienlogik der Anwendung festzulegen, bevor mit der eigentlichen Implementierung begonnen wurde. Die Mockups dienen dabei als konzeptionelle Grundlage für die spätere Umsetzung und ermöglichen eine frühzeitige Absprache der Benutzerführung mit dem Auftraggeber sowie der Anordnung der einzelnen Bedienelemente. Insbesondere wurde darauf geachtet, die Benutzeroberfläche übersichtlich und intuitiv zu gestalten, um eine einfache Bedienung für alle Benutzergruppen zu gewährleisten.

Für jede zentrale Benutzerrolle (Besucher*innen, Standpersonal) wurden entsprechende Oberflächen entworfen. Dabei wurden typische Nutzungsszenarien berücksichtigt, wie beispielsweise das Aufgeben einer Bestellung, die Anzeige von Bestellstatus oder die Verwaltung von Bestellungen

durch das Standpersonal. Ein besonderer Fokus lag auf der klaren Darstellung von Statusinformationen, wie beispielsweise dem Fortschritt einer Bestellung. Darüber hinaus wurden wichtige Funktionen wie das Stornieren von Bestellungen oder das Aufladen von Guthaben leicht zugänglich gestaltet. Die im Entwurfsprozess erstellten Mockups sind im Anhang unter [A.4: Mockups](#) einsehbar und dienten als Vorlage für die spätere programmatische Umsetzung. Während der Implementierungsphase wurden kleinere Anpassungen vorgenommen, um technische Gegebenheiten sowie praktische Anforderungen zu berücksichtigen. Durch den Einsatz von Mockups konnte die Benutzeroberfläche bereits vor der Implementierung strukturiert geplant und potenzielle Usability-Probleme frühzeitig erkannt werden.

4.4 Datenmodell

Im Rahmen der Entwurfsphase wurde ein Datenmodell entwickelt, um die für das System benötigten Informationen strukturiert und konsistent in einer relationalen Datenbank abzubilden. Ziel war es, alle relevanten Datenobjekte des Bestell- und Abholprozesses sowie deren Beziehungen zueinander nachvollziehbar zu modellieren. Als Grundlage für die Modellierung wurde zunächst ein Entity-Relationship-Diagramm erstellt. Dieses dient der konzeptionellen Darstellung der wichtigsten Entitäten, Attribute und Beziehungen. Aufbauend darauf wurde ein relationales Datenbankmodell entworfen, welches die technische Umsetzung in Microsoft SQL Server beschreibt. Beide Darstellungen sind im Anhang unter [A.5: ER-Diagramm](#) und [A.6: Relationales Datenbankmodell](#) einsehbar.

Das Datenmodell bildet die zentralen Bestandteile des Systems ab. Dazu zählen insbesondere die Entitäten *Tickets*, *Orders*, *OrderPositions*, *Products*, *Stands*, *Status* und *Messages*. Ergänzt werden diese durch Verknüpfungstabellen wie *Stand2Product* und *Order2Ticket*, welche n:m-Beziehungen zwischen den jeweiligen Entitäten auflösen.

Bei der Modellierung wurde darauf geachtet, Redundanzen weitgehend zu vermeiden und Beziehungen zwischen den Entitäten über Primär- und Fremdschlüssel eindeutig abzubilden. Das Datenbankmodell befindet sich in der dritten Normalform. Durch die relationale Struktur können Daten konsistent gespeichert und effizient abgefragt werden. Dies bildet die Grundlage für die zuverlässige Verarbeitung von Bestellungen, Beständen und Benutzerinformationen innerhalb der Anwendung.

4.5 Geschäftslogik

Um die Umsetzung der Geschäftsobjekte vorab grob planen zu können, wurde während der Entwurfsphase ein Klassendiagramm erstellt. Es macht den ungefähren Aufbau der zu implementierenden Objektklassen anschaulich und ist im Anhang unter [A.7: Klassendiagramm](#) zu finden.

Mit Blick auf die in Abschnitt [4.2: Architekturdesign](#) erwähnten Rollen des MVC-Musters, nimmt die Klasse *Database* die Rolle des Models ein. Die jeweiligen Seitenobjekte (*LoginPage*, *VisitorPage*, *OrderPage*, *SellerPage*) bilden die Controller, welche die zugrunde liegende Logik hinter den View-Objekten bereitstellen. Über die jeweilige *get_page()* Methode wird innerhalb der Controller-Klassen das, auf Tkinter aufgebaute, View-Objekt initialisiert und mit dem Controller verbunden.

Für den Dialog zum aufladen des Ticket-gebunden Guthabens (*CreditDialog*) wird eine eigene, wiederverwendbare Klasse implementiert. So kann der Dialog einfach und erweiterbar an diversen Stellen der Anwendung eingebunden werden.

4.6 Maßnahmen zur Qualitätssicherung

Es werden Black-Box-Tests durch eine Test-Gruppe bestehend aus Mitschülern geplant. Die Testfälle werden aus dem zur Verfügung gestellten Anforderungsdokument erstellt. In Folge der Durchführung der Tests werden Fehler analysiert und korrigiert. Es werden kontinuierliche Feature-Tests durch die Entwickler während der Entwicklung durchgeführt.

4.7 Pflichtenheft

Am Ende der Entwurfsphase wurde ein Pflichtenheft erstellt. Dieses baut auf dem Lastenheft (Abschnitt [3.6: Lastenheft/Fachkonzept](#)) auf und beschreibt wie und womit die Autoren die Anforderungen des Fachbereiches umsetzen möchten. Es dient als Grundlage für die Realisierung des Projektes. Ein Auszug aus dem Pflichtenheft ist im Anhang [A.8: Pflichtenheft \(Auszug\)](#) einzusehen.

5 Implementierungsphase

5.1 Implementierung der Datenstrukturen

Auf Grundlage der in Abschnitt [4.4: Datenmodell](#) definierten Datenstruktur wurde die Implementierung der Datenbank vorgenommen. Zunächst wurde eine neue Microsoft SQL Server Instanz initialisiert und gehostet. Über das SQL Server Management Studio wurde innerhalb der Server-Instanz eine neue Datenbank angelegt. Dieser wurden die benötigten Tabellen (und Demo-Daten) durch SQL-Statements hinzugefügt. Die erstellte Datenbank entspricht somit der in der Entwurfsphase definierten Struktur und erfüllt die nötigen Anforderungen.

5.2 Implementierung der Geschäftslogik

Da die Implementierung der Geschäftslogik den Kernbestandteil des gesamten Projektes darstellt, soll diese im folgenden Abschnitt genauer erläutert werden. Um die Implementierung möglichst komfortabel durchführen zu können, ist eine geeignete Entwicklungsumgebung von großer Bedeutung. Die Autorin hat dafür Visual Studio Code verwendet.

Zu Beginn der Implementierung wurde das Grundgerüst der Anwendung erstellt. Dafür wurde ein Main-File erstellt, welches als Einstiegspunkt der Ausführung der Anwendung dient. Dort werden Fonts, die Skalierung und die Initialisierung der einzelnen Objekt-Klassen definiert. Der Main-Loop des Wurzelobjekts der Tkinter-Objekte wird ebenso hier definiert. Als nächstes wurde die Klasse *Database* implementiert, welche alle benötigten Funktionen zur Kommunikation mit der Datenbank mit Hilfe des pyodbc-Frameworks bereitstellt. Anschließend haben die Autoren sich mit der Implementierung der einzelnen Seiten-Objekten befasst und diese mit den benötigten Funktionen versorgt, welche von den späteren View-Objekten der Tkinter-Oberfläche verwendet werden. Eine lokale Generierung der QR-Codes für den Abholprozess wurde über die Python Bibliothek *qrcode* realisiert. Als Grundlage der Generierung werden Informationen abgerufen, welche in den Daten des jeweiligen Tickets innerhalb der Datenbank hinterlegt werden. Als letztes wurde eine fortlaufende Aktualisierung der Wartezeiten sowie der Nachrichtenobjekte innerhalb der Model-Daten der Besucherseite implementiert.

5.3 Implementierung der Benutzeroberfläche

Die Implementierung der Benutzeroberfläche wurde, wie im Abschnitt [4.2: Architekturdesign](#) geplant, mit Hilfe des Tkinter-Frameworks umgesetzt. Für das Design der Oberfläche haben sich die Autoren an den in der Entwurfsphase entworfenen Mockups orientiert. Für die Darstellung von Bild-Objekten wie dem generierten QR-Code wurde das Pillow-Framework (PIL) verwendet. Es wurde eine globales Font festgelegt, was über alle Oberflächenobjekte hinweg verwendet wird. Auch wurde eine grobe Skalierung implementiert, um die Bedienung der Anwendung Bildschirmgrößen-unabhängig zu ermöglichen. Die Platzierung der Oberflächenelemente wird über ein Grid-Layout gesteuert.

Die gesamte Implementierung der Geschäftslogik und der Benutzeroberfläche erfolgte nach dem in Abschnitt [2.3: Entwicklungsprozess](#) beschriebenen Ablauf. Der Code wurde fortlaufend durch die Autoren gereviewed und während der Entwicklung funktional getestet. Die Implementierungsphase konnte mit dem Fertigstellen der Benutzeroberfläche abgeschlossen werden.

6 Abnahmephase

Nach Fertigstellung der gesamten Anwendung, wurde das Projekt den Lehrkräften zur Endabnahme vorgelegt. Bereits während der Entwicklung stand den Lehrkräften über das GIT-

Repository eine Einsicht in den Entwicklungsstand zur Verfügung. Die Anwendung wurde erfolgreich durch die Testgruppe getestet und gefundene Fehlfunktionen korrigiert. Abschließend fanden eine Präsentation der Anwendung sowie ein anschließendes Fachgespräch mit den Lehrkräften statt.

7 Dokumentation

Die Dokumentation des Projektes besteht aus drei Bestandteilen: der Projektdokumentation, dem Benutzerhandbuch und der Entwicklerdokumentation. In der Projektdokumentation beschreiben die Autoren die einzelnen Phasen, die während der Umsetzung des Projektes durchlaufen wurden.

Das Benutzerhandbuch beschreibt den Aufbau und die Funktionsweise der Anwendung. Es soll Anwendern als Anhaltspunkt für Nachfragen zur Verfügung stehen und einen einfachen, sowie schnellen Einstieg für Festivalbesucher ermöglichen. Das Handbuch ist im Anhang unter [A.9: Benutzerhandbuch](#) zu finden.

Die konkrete Entwicklerdokumentation wurde im Rahmen dieses Demoprojektes weggelassen.

8 Fazit

8.1 Soll-/Ist-Vergleich

Bei einer rückblickenden Betrachtung des Demo-Abschlussprojektes, kann festgehalten werden, dass alle zuvor festgelegten Anforderungen gemäß dem Pflichtenheft erfüllt wurden. Der während der Planungsphase erstellte Projektplan(Abschnitt [2.1: Projektphasen](#)) konnte eingehalten werden. Es mussten aufgrund von Feedback zusätzliche Anforderungen während der Entwicklung umgesetzt werden, dies hat aber den Projektverlauf aufgrund geplanter Pufferzeiten nicht beachtenswert beeinflusst.

8.2 Lessons Learned

Im Zuge des Projektes konnte die Autoren wertvolle Erfahrungen bezüglich der Planung und Durchführung von Projekten sammeln. Auch wurde deutlich wie wichtig eine sinnvolle Planung ist und welche Einflüsse neue Anforderungen oder Änderungen von Anforderungen während der Entwicklung haben können. Die Skalierung der View-Objekte innerhalb des Tkinter-Frameworks gestaltete sich zu Beginn etwas schwierig. Ebenso war die Bereitstellung einer Datenbankanbindung über verschiedene Endgeräte ein erstes Hindernis. Abschließend lässt sich sagen, dass das Demo-Projekt als gute Übung für das eigentliche IHK-Abschlussprojekt diene.



8.3 Ausblick

Obwohl alle im Lastenheft definierten Anforderungen realisiert werden konnten, gibt es durchaus Ansätze für zukünftige Erweiterungsmöglichkeiten. Es könnte eine Mobile Payment Integration erweitert werden. Ebenso könnte über eine KI-gestützte Nachfrageprognose nachgedacht werden, welche einen strukturierten Nachfüllprozess ermöglichen würde. Die Anwendung könnte als Multi-Festival-Plattform erweitert werden, um Funktionen nicht nur Festival-spezifisch bereitzustellen.



A Anhang

A.1 Verwendete Ressourcen

Hardware

- Lenovo Thinkpad 16p

Software

- Windows 11 - Betriebssystem
- Visual Studio Code - Entwicklungsumgebung
- Python (Bibliotheken: Tkinter, pyodbc)
- Microsoft SQL Server - Datenbank
- SQL Server Management 22 - Datenbanksystem
- git / Github - Versionsverwaltung
- Microsoft Office Professional Plus 2024
- MiKTeX – Dokumentation via LATEX

Personal

- Drei Entwickler - Zur Umsetzung des Projektes
- Testgruppe zusammengesetzt aus Mitschülern
- Lehrer - Auftraggeber / Abnahme des Projektes



A.2 Lastenheft (Auszug)

Es folgt ein Auszug aus dem Lastenheft mit Fokus auf die Anforderungen:

A.2.1 Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben die konkreten Funktionen, die das System bereitstellen muss.

Benutzerverwaltung

- Anmeldung von Besucher*innen mittels Ticketnummer
- Verwaltung eines persönlichen Guthabens
- Unterscheidung von Benutzerrollen (Besucher*innen, VIP-Besucher*innen, Standpersonal)
- Generierung eines eindeutigen QR-Codes pro Besucher-Ticket

Bestellsystem

- Anzeige verfügbarer Verkaufsstände und Produkte
- Darstellung von Produktverfügbarkeiten und geschätzten Wartezeiten
- Hinzufügen von Sonderwünschen pro Bestellung
- Priorisierung von Bestellungen für VIP-Besucher*innen
- Durchführung von Bestellungen mit Guthabenabbuchung

Bestellverwaltung

- Anzeige aller Bestellungen für Besucher*innen inklusive Status
- Verwaltung von Bestellungen durch Standpersonal
- Änderung des Bestellstatus (Bestellung aufgeben, In Bearbeitung, Bereit zur Abholung, Ausgegeben)

Abholprozess

- Anzeige des QR-Codes zur Abholung
- Verifikation der Bestellung durch Scannen des QR-Codes
- Abschluss der Bestellung nach erfolgreicher Ausgabe

Bestandsverwaltung

- Verwaltung der Produktbestände pro Stand
- Automatische Reduzierung des Bestands nach Bestellung oder Ausgabe
- Kennzeichnung nicht verfügbarer Produkte

Stornierung und Sonderfälle

- Stornierung von Bestellungen bei fehlender Verfügbarkeit
- Automatische Rückerstattung des Guthabens
- Behandlung nicht abgeholter Bestellungen

A.2.2 Nichtfunktionale Anforderungen

Die nichtfunktionalen Anforderungen beschreiben qualitative Eigenschaften des Systems.

Benutzbarkeit (Usability)

- Intuitive und übersichtliche Benutzeroberfläche
- Schnelle Erlernbarkeit ohne umfangreiche Einarbeitung
- Klare Darstellung von Statusinformationen und Rückmeldungen

Leistungsanforderungen (Performance)

- Schnelle Verarbeitung von Bestellungen in Echtzeit
- Geringe Ladezeiten bei Benutzerinteraktionen
- Stabiler Betrieb auch bei hoher Anzahl gleichzeitiger Benutzer



Zuverlässigkeit

- Korrekte Verarbeitung von Bestellungen ohne Datenverlust
- Konsistente Speicherung von Bestell- und Bestandsdaten

Sicherheit

- Schutz vor unbefugtem Zugriff auf Benutzerdaten
- Sichere Verarbeitung von Guthaben und Transaktionen

Wartbarkeit und Erweiterbarkeit

- Modularer Aufbau des Systems
- Möglichkeit zur Erweiterung um zusätzliche Funktionen (z. B. weitere Stände)

A.3 Use-Case-Diagramm

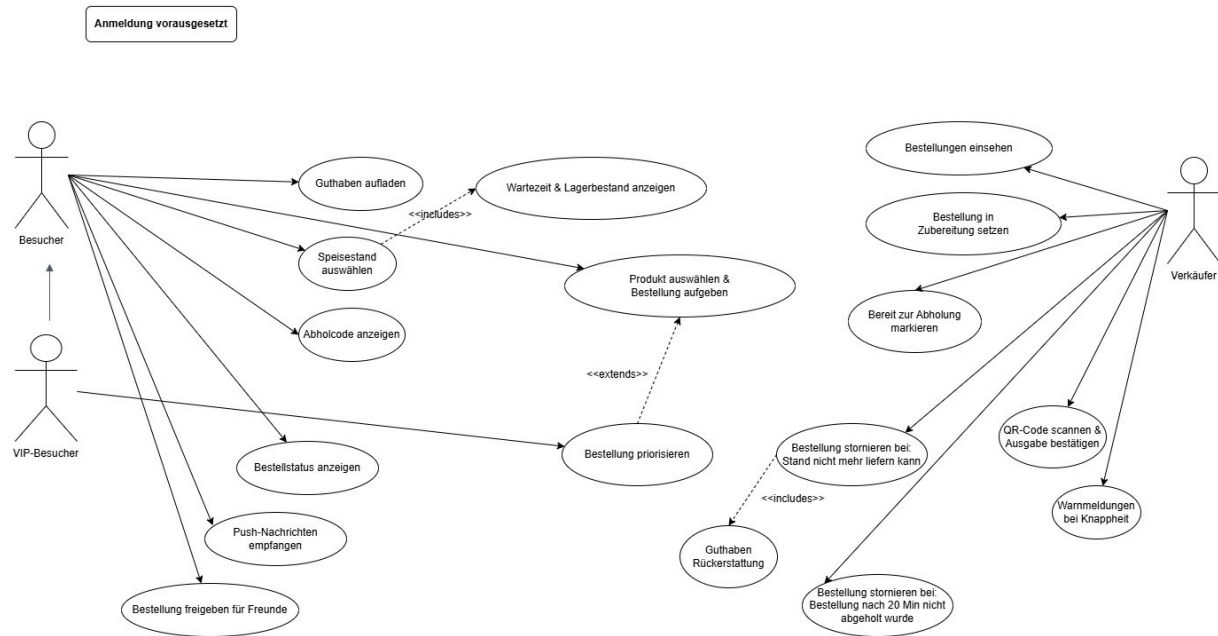


Abbildung 1: Use-Case-Diagramm

A.4 Mockups

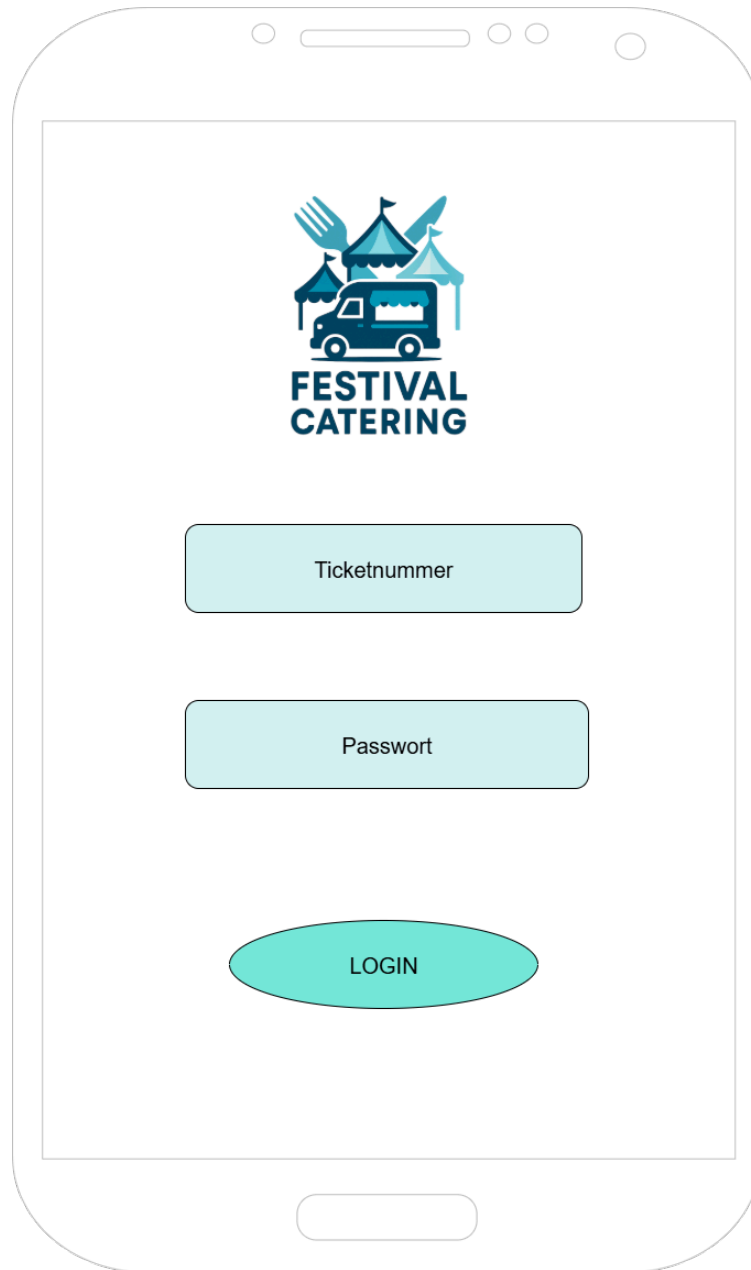


Abbildung 2: Login-Seite

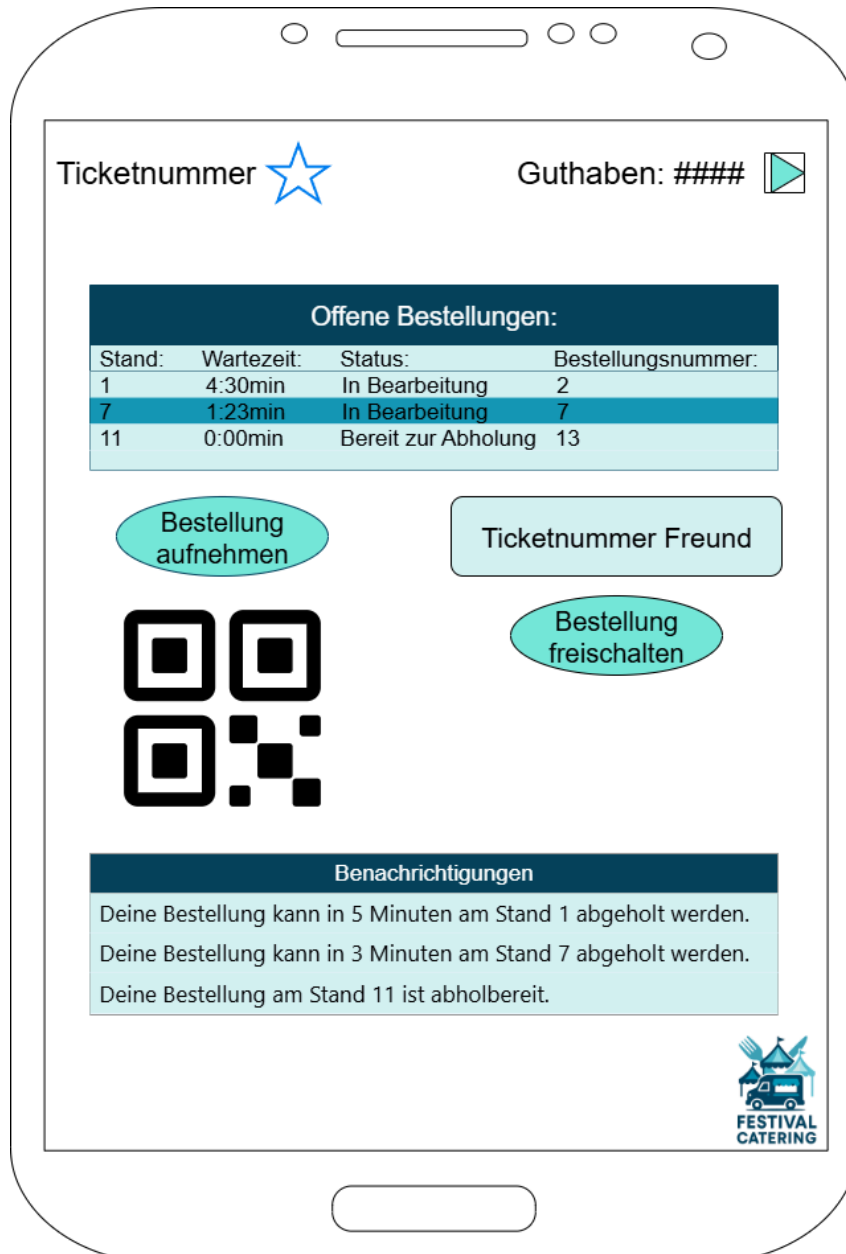
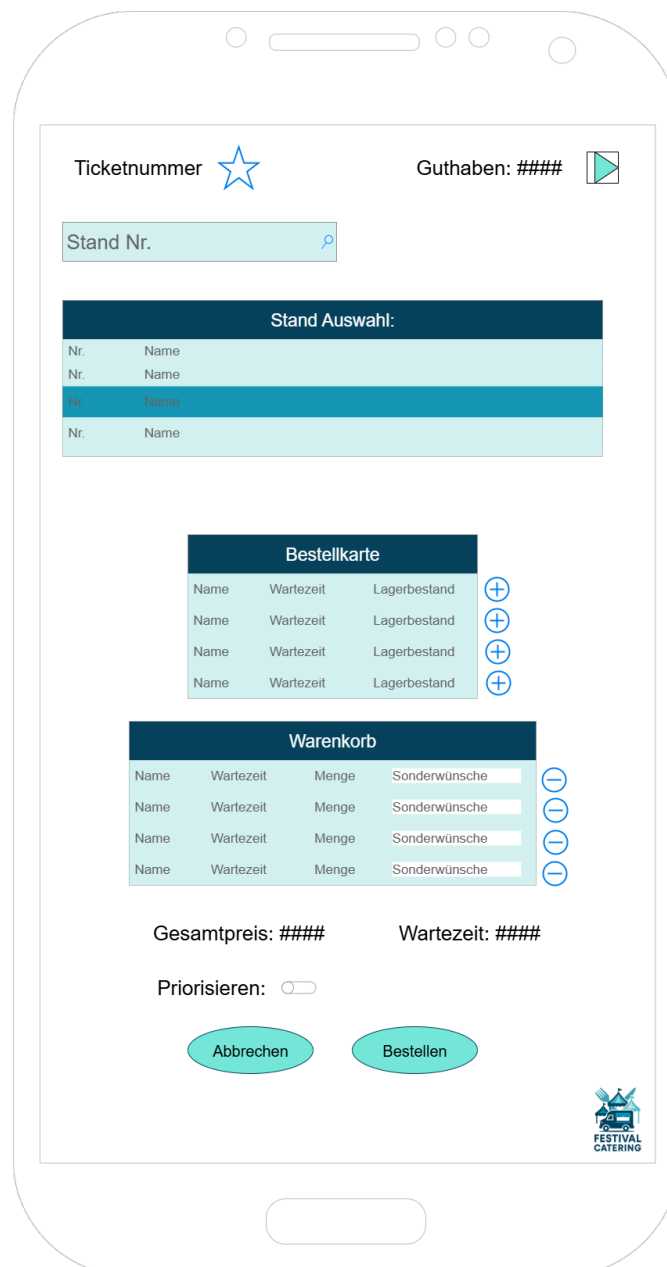


Abbildung 3: Besucher-Seite



The image shows a mobile app interface for 'Festival Catering'. At the top, there's a header with 'Ticketnummer' and a star icon, and 'Guthaben: ####' with a play button icon. Below this is a search bar labeled 'Stand Nr.' with a magnifying glass icon. The main content area is divided into three sections: 'Stand Auswahl:', 'Bestellkarte', and 'Warenkorb'. The 'Stand Auswahl:' section contains a table with 4 rows, each with 'Nr.' and 'Name' columns. The 'Bestellkarte' section contains a table with 4 rows, each with 'Name', 'Wartezeit', and 'Lagerbestand' columns, and a '+' button to the right of each row. The 'Warenkorb' section contains a table with 4 rows, each with 'Name', 'Wartezeit', 'Menge', and 'Sonderwünsche' columns, and a '-' button to the right of each row. At the bottom, there are two buttons: 'Abbrechen' and 'Bestellen'. The bottom right corner features the 'Festival Catering' logo.

Ticketnummer  Guthaben: #### 

Stand Nr. 

Stand Auswahl:

Nr.	Name
Nr.	Name
Nr.	Name
Nr.	Name

Bestellkarte

Name	Wartezeit	Lagerbestand	
Name	Wartezeit	Lagerbestand	
Name	Wartezeit	Lagerbestand	
Name	Wartezeit	Lagerbestand	

Warenkorb

Name	Wartezeit	Menge	Sonderwünsche	
Name	Wartezeit	Menge	Sonderwünsche	
Name	Wartezeit	Menge	Sonderwünsche	
Name	Wartezeit	Menge	Sonderwünsche	

Gesamtpreis: #### Wartezeit: ####

Priorisieren: ☐

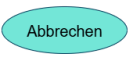
 



Abbildung 4: Bestells-Seite

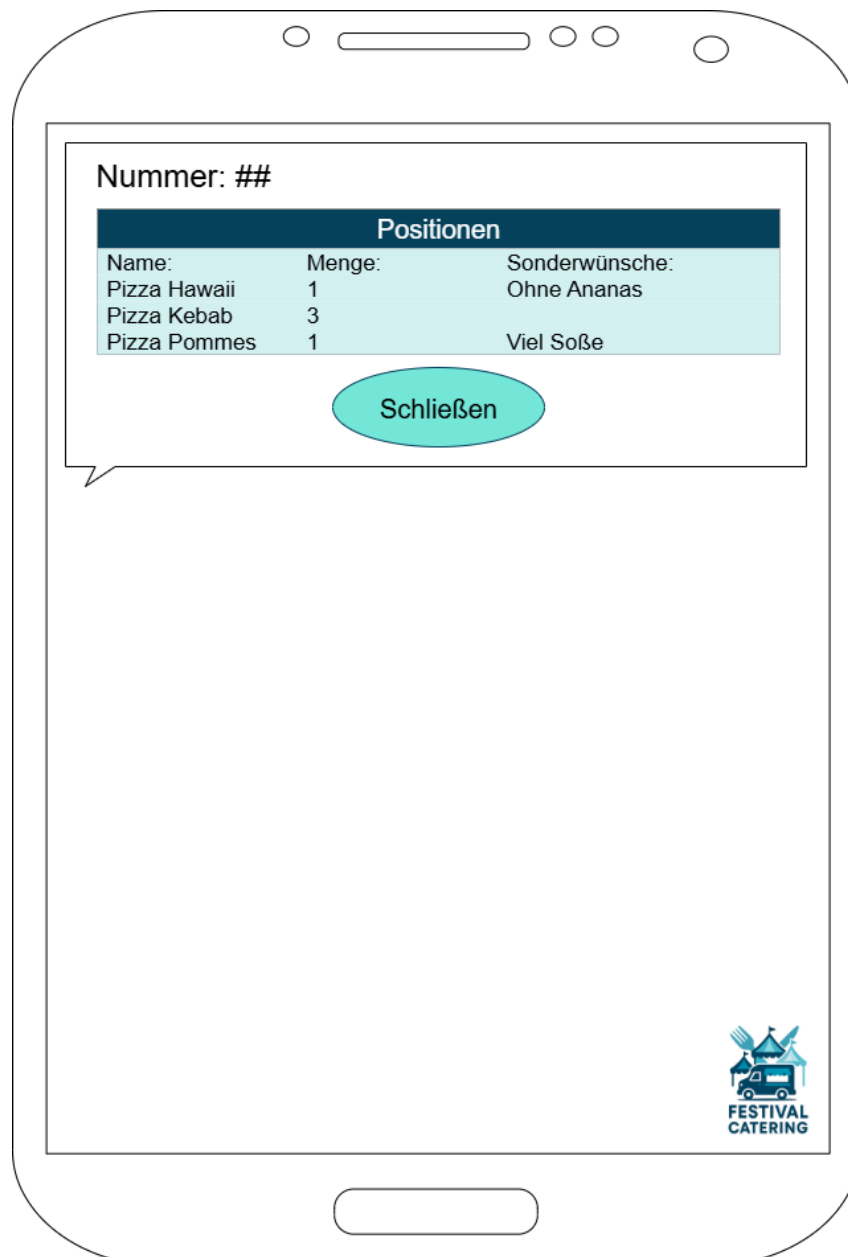


Abbildung 5: Einzelanzeige Bestellung

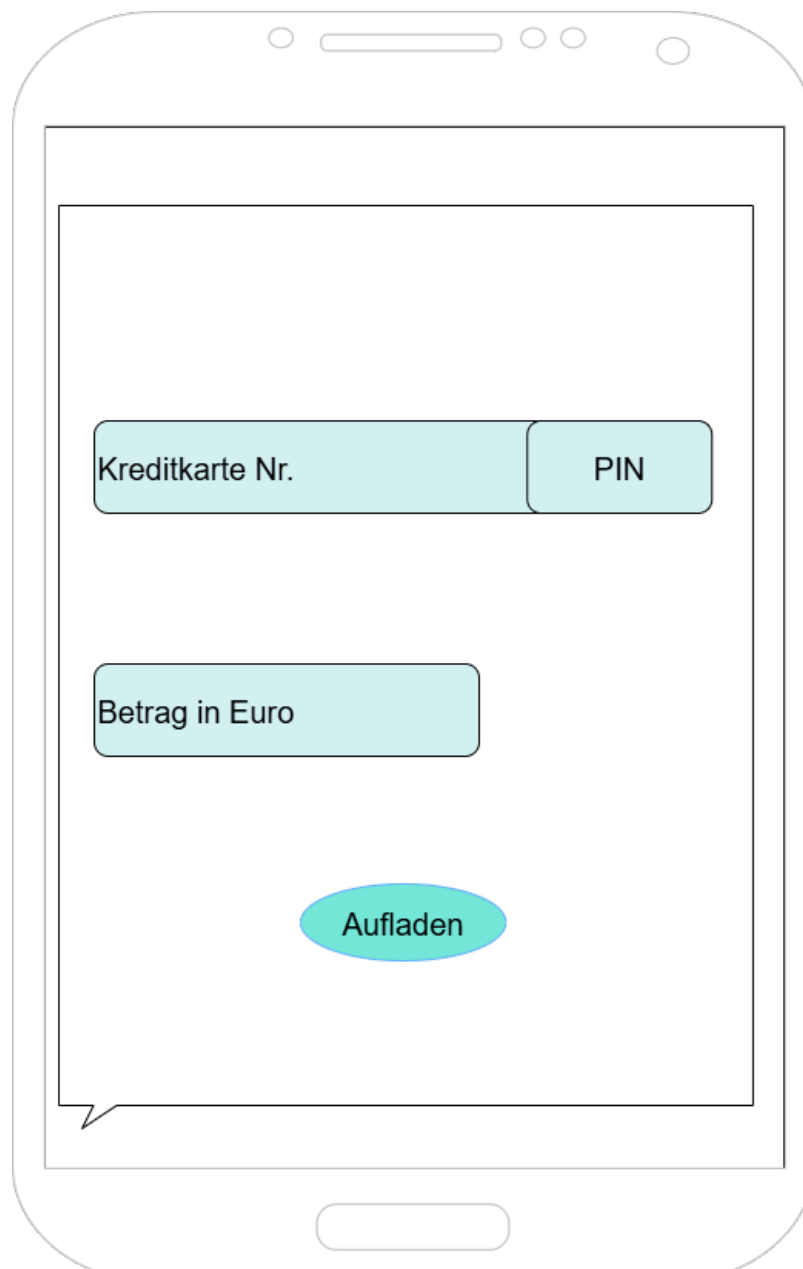


Abbildung 6: Guthaben Popup



Abbildung 7: Verkäufer-Seite

A.5 ER-Diagramm

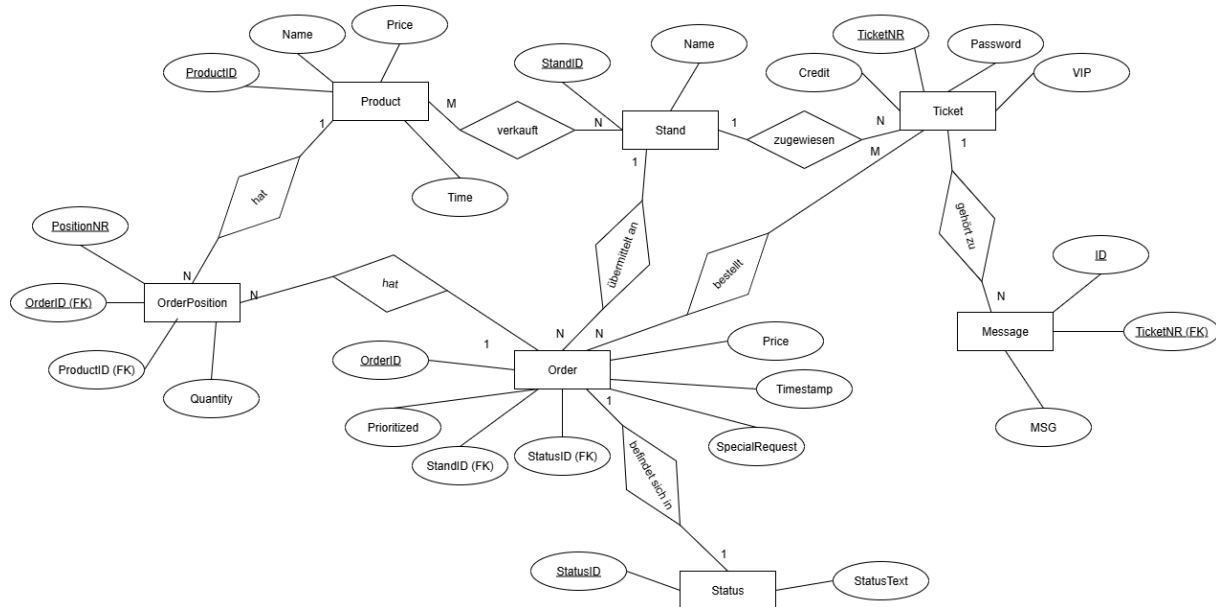


Abbildung 8: ER-Diagramm

A.6 Relationales Datenbankmodell

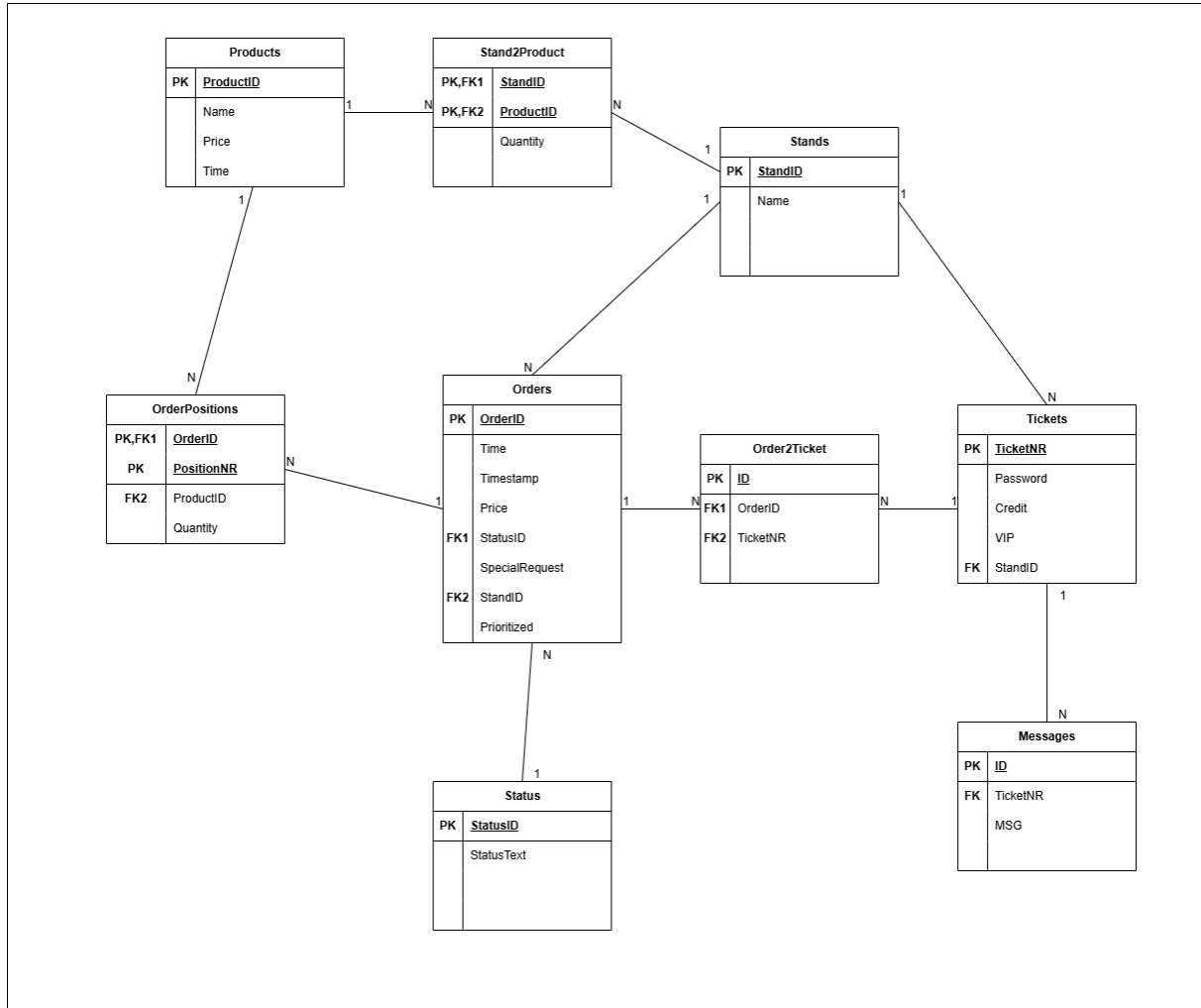


Abbildung 9: Relationales Datenbankmodell

A.7 Klassendiagramm

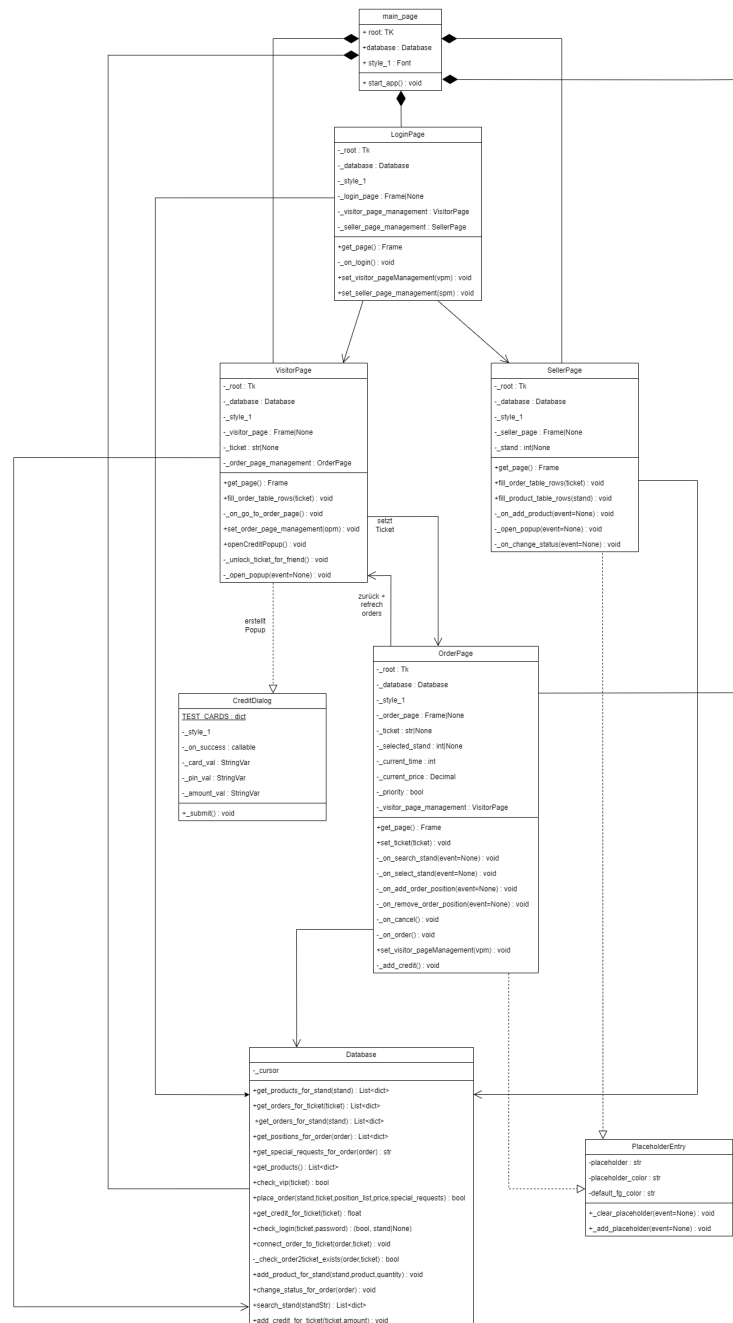


Abbildung 10: Klassendiagramm

A.8 Pflichtenheft (Auszug)

In folgendem Auszug aus dem Pflichtenheft wird die geplante Umsetzung der im Lastenheft definierten Anforderungen beschrieben:

A.8.1 Umsetzung Funktionale Anforderungen

Benutzerverwaltung

- Die Anmeldung erfolgt über die Eingabe einer Ticketnummer und eines Passworts
- Die Benutzeroberfläche ist als Login-Maske mit zwei Eingabefeldern und einem Login-Button umgesetzt
- Validierung der Eingaben gegen die Datenbank
- Zuordnung einer Benutzerrolle (Besucher, VIP, Standpersonal)
- Anzeige des aktuellen Guthabens nach erfolgreichem Login
- Button zum öffnen eines Dialogs zum aufladen des Guthabens
- QR-Code wird pro Ticket lokal aus zugrunde liegenden Daten aus dem Model generiert

Bestellsystem

- Anzeige aller verfügbaren Verkaufsstände in einer Liste
- Auswahl eines Standes über Suchfeld oder Tabellenansicht
- Anzeige der Produkte inklusive:
 - Name
 - Wartezeit
 - Lagerbestand
- Produkte können über Buttons dem Warenkorb hinzugefügt werden
- Positionen im Warenkorb können über Buttons entfernt werden
- Der Bestellung kann über ein Eingabefeld ein Sonderwunsch hinzugefügt werden
- Über eine Anzeige werden Gesamtpreis und Gesamtwartezeit angezeigt
- VIP-Besucher können über einen Schalter die Bestellung als priorisiert markieren
- Per Button kann die Bestellung aufgegeben oder abgebrochen werden



- Nach Abschluss wird das verfügbare Guthaben validiert und aktualisiert

Bestellverwaltung

- In der Besucherübersicht werden alle laufenden Bestellungen angezeigt
- Folgende Felder werden in der Tabellensicht dargestellt:
 - Standnummer
 - Wartezeit
 - Status
 - Bestellnummer
- Das Standpersonal kann die, dem Stand zugehörigen, Bestellungen in einer Tabelle innerhalb der Verkäufer-Sicht einsehen
- Über die Verkäufer-Sicht kann der Status der Bestellungen (Bestellung aufgeben, In Bearbeitung, Bereit zur Abholung, Ausgegeben) angepasst werden

Abholprozess

- Der generierte QR-Code wird innerhalb der Besucher-Sicht angezeigt
- Verifikation der Bestellung durch Scannen des QR-Codes
- Validierung der Bestellung im System
- Statusänderung auf „Ausgegeben“ durch das Standpersonal

Bestandsverwaltung

- Die Bestände werden automatisch durch das System verwaltet
- Reduzierung des Lagerbestands bei Bestellung
- Aktualisierung bei Ausgabe
- Kennzeichnung von Produkten mit einer Warnung innerhalb der Verkäufer-Sicht als bei Unterschreitung eines Grenzwerts

Stornierung und Sonderfälle

- Automatische Stornierung bei nicht ausreichendem Bestand
- Bestellung kann durch Besucher storniert werden
- Automatische Rückerstattung des Guthabens
- Stornierung nicht abgeholter Bestellungen kann durch Standpersonal durchgeführt werden

A.8.2 Umsetzung Nichtfunktionale Anforderungen

Benutzbarkeit (Usability)

- Klare Strukturierung in Bereiche (Bestellungen, Warenkorb, Benachrichtigungen)
- Konsistente Farbgestaltung
- Intuitive Bedienung durch Buttons und Tabellen

Leistungsanforderungen (Performance)

- Datenbankabfragen werden optimiert ausgeführt
- Minimierung von Ladezeiten durch direkte Verarbeitung
- Echtzeit-Aktualisierung von Bestellstatus und Beständen

Zuverlässigkeit

- Korrekte Verarbeitung von Bestellungen ohne Datenverlust
- Bestell und Bestandsdaten werden über die Datenbank gespeichert und direkt abgerufen

Sicherheit

- Authentifizierung über Ticketnummer und Passwort
- Zugriffsbeschränkung je nach Benutzerrolle
- Validierung aller Eingaben



Wartbarkeit und Erweiterbarkeit

- Modularer Aufbau (Trennung von GUI, Logik und Datenbank)
- Erweiterbarkeit für zusätzliche Funktionen (z. B. weitere Rollen oder Zahlungsarten) kann einfach über die Datenbank durchgeführt werden



A.9 Benutzerhandbuch